

Rendszerterv

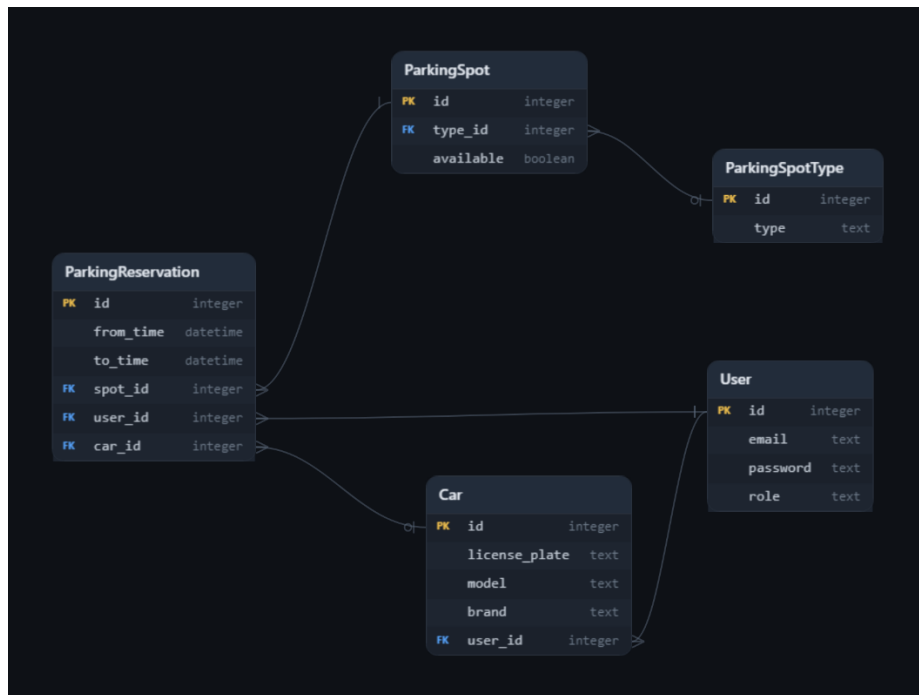
Az alkalmazás egy Restful elvekre épülő Web API, amely ellátja egy parkoló/parkolóház foglalásához szükséges adatok kezelését.

Stack

Adatbázis	SQLite
ORM	Entity Framework
Keretrendszer	C# / ASP.NET Core Web API .NET 10
Konténerizáció	Docker

Adatbázis

Az adatbázis SQLite, az alábbi képen látható táblákkal:



Az adatbázisban az idegenkulcsok be lettek állítva.

- Car: Felhasználókhöz van hozzárendelve, egy autót több felhasználó is felvehet. Információval szolgál a beengedéshez.
- User: Felhasználók adatait tartalmazza: Email, Role, Jelszó (Hash)
- ParkingSpotType: Parkolóhely típusokat tartalmaz. (korlátozott hozzáférésű helyek)

- ParkingSpot: Tényleges parkolóhelyeket tartalmazza, minden rekord egy parkolóhelynek felel meg, a továbbfejleszthetőség érdekében.
- ParkingReservation: Tartalmazza a foglalás adatait.

Adat réteg

Az adatok eléréséhez Entity Frameworkot használok, Database first megközelítéssel. A Connection String az appsettings.json fájlban található. (Only for development)

Az adatok strukturálása és könnyebb kezelhetősége érdekében Data Transfer Object, DTO-kat hoztam létre.

Szolgáltatás réteg

- PasswordService: A jelszavak biztonságos tárolásáért, hashelésért és validálásáért felel.
- TokenService: JWT tokenek generálását és validálását végzi.

Az alkalmazás használatához regisztrálni és belépni szükséges. Belépés után az alkalmazás generál egy JWT tokenet, amely tartalmazza az alábbiakat: userID, role, email. Az alkalmazás fejlesztése során figyeltem arra, hogy userid-t csak validált tokenből használjak fel, például az adatok szűrésére. Az alkalmazásban kettő darab szerepkör van:

- User: Tud regisztrálni/belépni, új autót fölvenni, és parkolót foglalni.
- Admin: Parkolóhelyeket, azok típusát tudja módosítani, törölni, létrehozni.

Vezérlő réteg

A controllereket úgy alakítottam ki, hogy jól elkülöníthető funkcionalitással bírjanak.

Az alábbi controllerek találhatóak meg az alkalmazásban:

- CarsController
- ReservationsController
- SpotController
- UserController

Tesztelés

A solution tartalmaz egy teszteléshez való projektet (NUnit). A szolgáltatásréteg összes metódusa le lett tesztelve, a controllerek és metódusai manuálisan lettek tesztelve.

Futtatás

Az alkalmazás egyszerűen futtatható egy docker-compose up paranccsal.

Továbbfejlesztési lehetőség

- Refresh Token: Azért szükséges, hogy a felhasználó a generált token lejárta után is tudja használni az alkalmazást.
- Email küldése: Jelszó megváltoztatásához, foglalás visszaigazolásához.
- Parkolóhelyek fizikai helyének a mentése
- Endpoint (API key-el ellátva), a sorompós, rendszám alapú automatikus beegedéshez.

Reflexió

Maga a feladat érdekes volt logikailag a legnagyobb kihívást a dátumok kezelése okozta. Megpróbáltam olyan formában visszaadni az adatokat, hogy az frontend oldalon egyszerűen kezelhető legyen. További nehézségeket okozott a Docker, már régen használtam, valamint NUnit, ami teljesen új volt a számomra. A feladatot 4 és fél óra alatt sikerült megcsinálnom a dokumentáció nélkül. Ami előnyt jelentett számomra, az a keretrendszer függetlenség volt, a .NET-et választottam, mert abban már sok API-t fejlesztettem.